

2. Sent data on the connection is not acknowledged at the TCP layer, and the configured **TCP User Timeout** expires.

When the TCP layer of a node gets notified that the peer has closed the connection, it SHALL close its end of the connection as well, and notify the application. Subsequently, all active **Exchanges** over that connection SHOULD also be closed as they would be unusable over a closed connection.

Either side MAY choose to re-establish the connection when it is closed or determined to be broken. A node SHOULD back-off (typically, based on a Fibonacci back-off sequence) a random amount of time after the connection closure, before attempting to establish the connection again.

A node, upon receipt of a connection request from a peer, SHOULD discard its own backed-off connection retry timer to the same peer, if one is active.

This random back-off mechanism SHOULD prevent connection races between peers for most, if not all scenarios. In addition, nodes SHOULD try to reap old unused connections as much as possible to conserve resources.

4.15.2.3. Memory Requirements for Large messages

Since the TCP transport is designed for large messages, the system platform is expected to have memory available for storing the received messages. The system platform MAY configure a **Maximum Message Size** for the payload that it is capable of receiving over the TCP connection. If a node receives a message header that indicates that the message is larger than the **Maximum Message Size** that it supports, then it SHALL close the connection, and SHOULD send a **Status Report** error message with a status code set to **MESSAGE_TOO_LARGE** back to the sender, before closing the connection.

Given that a node MAY be using both MRP and TCP for sending/receiving messages with different peers, and these transports have different **Maximum Message Size** configurations, the system SHOULD be able to dynamically change this configuration based on what transport the current Exchange is using.

4.16. Group Communication

This section specifies the semantics of sending and receiving multicast group messages and the lifecycle of such groupcast sessions. Multicast addressing is accomplished using the 16-bit **Group ID** field as the destination address. A multicast group is a collection of Nodes, all registered under the same multicast Group ID. A multicast message is sent to a particular destination group and is received by all members of that group.

4.16.1. Groupcast Session Context

Groupcast sessions are conceptually long-running, lasting the duration of a node's membership in a group. Group membership is tracked in the **Group Key Management Cluster**. However, on ingress of each groupcast message, the following ephemeral context SHALL be constructed to inform upper layers of groupcast message provenance:

1. **Fabric Index**: Records the local **Fabric Index** for the Fabric to which an incoming message's group is scoped.